Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Dersi dersin hocası Dr. Ahmet Elbir anlatıyor (Dev-C++ ekranındaki kullanıcı yolu `C:\Users\AHMETELBIR\...`). Ders 30 Nisan 2025 tarihli. Konular: bağlı listeler (linked list) ile kod yazma, yapı hizalaması (structure alignment), bit alanları (bit field), birlikler (union) ve özyineleme (recursion).
- Bağlı listeye kısa tekrar
  - Her düğüm (node) bir veri (data) ve bir sonraki düğümün adresini tutan bir işaretçi (`next`) içerir.
  - İlk düğümün işaretçisi `head` değişkeninde tutulur. `head` bilinirse bütün düğümlere sırayla erişilir.
  - Son düğümün `next` alanı `NULL` olur (tek yönlü, dairesel olmayan liste).
  - Sona hızlı eleman eklemek için son düğüm ayrıca `tail` işaretçisinde tutulabilir. Böylece her eklemede baştan sona gezmek gerekmez.
  - Son düğümün `next`'i `NULL` yerine `head`'i gösterirse dairesel (circular) tek yönlü liste olur.
  - Çift yönlü (doubly) listede her düğümde `next` ve `prev` olmak üzere iki işaretçi vardır. Dairesel çift yönlü listede `head`'in `prev`'i son düğümdür, ayrı `tail` tutmaya gerek kalmaz.
  - Liste boyutu baştan belli değildir. Bellek izin verdiği kadar düğüm eklenebilir.
- Düğüm yapısı
  - `typedef struct node { int data; struct node *next; } node;`
- Düğüm oluşturma: neden `malloc`?
  - Fonksiyon içinde `node n;` diye yerel bir düğüm tanımlayıp adresini listeye bağlamak yanlıştır. Yerel değişken stack bölgesindedir. Fonksiyon bitince o alan başka değişkenlere verilir ve düğüm "garanti" olmaktan çıkar.
  - Kalıcı düğümler heap bölgesinde olmalıdır. Bu yüzden düğümler dinamik bellek yönetimiyle (`malloc`) oluşturulur. "Bir bağlı liste oluşturuyorsak düğümleri kesinlikle dinamik bellek yönetimi metotlarıyla oluşacak."
  - `create_node` fonksiyonu kullanıcıdan veriyi alır, `malloc(sizeof(node))` ile düğüm açar, `data`'yı yazar, `next`'i `NULL` yapar ve işaretçiyi döndürür.
  - `next` alanına `NULL` atanmazsa bellekteki çöp değer bir adres gibi yorumlanır ve liste bozulur.
- Listeyi yazdırma (`printList`)
  - `while (head != NULL) { printf("%d ", head->data); head = head->next; }`.
  - Önce `NULL` kontrolü, sonra veriye erişim yapılmalı. Sıra ters olursa `NULL` bir işaretçinin alanına erişilir ve program çöker.
  - Liste boşsa ("empty list") yazdırıp `return` etmek iyi bir uygulamadır.
- Sona ekleme (`addLast`)
  - İlk denemede döngü `while (head != NULL)` ile kuruldu ve sonra `head = n` atandı. Bu çalışmaz: `head` `NULL` olana kadar gidildiğinde son düğümle bağlantı kaybolur. Yeni düğüm listeye bağlanmaz, ayrıca yalnızca yerel `head` değişir.
  - Doğrusu: `while (head->next != NULL) head = head->next;` ile son düğümde durmak, sonra `head->next = create_node(...)`.
  - Sona ekleme `main`'deki `head`'i değiştirmez. Başa ekleme (`addFirst`) veya baştan silme ise `head`'i değiştirir. Bu durumda yeni `head`'in `main`'e bildirilmesi gerekir.
  - Araya ekleme, mevcut düğüm ile sonraki düğüm arasında bağlantı kurmayı gerekti-

rir.

- Bütün listeyi silme (`deleteList`)
  - Kaç düğüm oluşturulduysa o kadar `free` gerekir.
  - Baştan silinir. Her silmeden önce sonraki düğüm geçici bir işaretçide (`tmp`) yedeklenir, yoksa listenin geri kalanı kaybolur: `tmp = head; head = head->next; free(tmp);`.
  - Her defasında sona gidip sondan silmek $n + (n-1) + ... + 1 \approx n^2$ işlem demektir, çok maliyetlidir.
  - Silme sonrası `main`'deki `head` hâlâ eski (serbest bırakılmış) adresi gösterir. Program "patladı". Çözüm: fonksiyon `NULL` döndürür ve `main`'de `head = deleteList(head);` yazılır. Ya da ikinci dereceden işaretçi (`node **head`) kullanılır. Başa ekleme ve baştan silmede ikinci dereceden işaretçiye mutlaka ihtiyaç vardır.
- Slayttaki çalışan (employee) örneği
  - Düğüm verisi iç içe bir yapı (nested structure): ad, soyad, kimlik numarası, maaş.
  - `scanf` ile `name` ve `surname` (karakter dizileri) okunurken `&` kullanılmaz, çünkü dizi adı zaten adres verir. `id` ve `salary` (skaler) için `&` gerekir.
  - Örnek `head`'i global değişken yapmıştır. Böylece hiçbir fonksiyon `head`'i parametre almadan kullanabilir ve güncelleyebilir. Hoca bunu tavsiye etmez: global değişkenin yan etkileri vardır, doğrusu parametre aktarmaktır.
  - `deleteElement(gunner)`: silinecek düğüm `head` ise `head = gunner->next`. Değilse silinecek düğümün bir öncekine kadar gidilir (`p->next != gunner`) ve `p->next = p->next->next` yapılıp düğüm `free` edilir.
  - `findElement(name)`: listede gezilir, isim `strcmp` ile karşılaştırılır (== string için çalışmaz), bulunursa düğüm, bulunamazsa `NULL` döner.
  - `insertAfter(p, q)`: `q->next = p->next; p->next = q;`. Bağlantıların sırası önemlidir.
  - Hoca ödevi: bu kodu global `head` olmadan, `head`'i parametre olarak aktararak yeniden yazın. Ekleme ve silmeleri kağıt üzerinde çizerek adım adım izleyin.
- Yapı hizalaması (structure alignment)
  - Derleyici yapı üyelerini belleğe hizalı yerleştirir ve aralarına boşluk (padding) koyabilir. Bu yüzden `sizeof` bir yapı için üyelerin toplamından büyük çıkabilir.
  - Üyelerin sırasını değiştirmek bellekten kazandırabilir. Hizalamayı değiştiren derleyici seçenekleri vardır ama her derleyicide çalışmaz.
- Bit alanları (bit field)
  - Bir üyenin kaç bit yer tutacağı belirtilir: `unsigned d : 5;`. Örneğin cinsiyet için 1 bit yeter.
  - Örnek: doğum günü 5 bit, ay 4 bit, yıl 11 bit. Üç değer tek bir `int` kadar yerde tutulur.
  - Bit alanlarında genellikle `unsigned` kullanılır. `signed` olursa en yüksek bit işaret biti olarak yorumlanır.
  - Bit alanının adresi alınamaz (`&` kullanılamaz). Bu yüzden `scanf` ile doğrudan okunamaz. Önce geçici bir değişkene okunur, sonra bit alanına atanır. Derleyici hatası: `cannot take address of bit-field`.
  - Bit alanlarından dizi oluşturulamaz. Bit alanı içeren bir yapının dizisi ise oluşturulabilir.
  - Girilen değerin geçerli aralıkta (ör. gün 1-31) olup olmadığı ayrıca kontrol edilmelidir.
- Birlikler (union)
  - Yapıya benzer, ama üyeler aynı bellek alanını paylaşır. Boyut en büyük üye kadardır.
  - Aynı anda yalnızca bir üye anlamlı veri tutar. Örnek: Türk vatandaşı için doğduğu şehir, yabancı için uyruk. İkisi aynı anda olamayacağı için 20 baytlık alan iki kez ayrılmaz.
  - Derste: `union { int a; char b; float c; }` için `sizeof` 4. `n.b = 'A'` sonrası `a` da 65 görünür. `n.a = 97` sonrası `b` küçük `a` olur, çünkü alanın tamamı yeniden yazılır.
  - İşaretçi bir yeri göstermeden kullanılamaz: `node *n; n->b = ...` yanlıştır. Önce `node l; n = &l;` veya `malloc` gerekir. Hocanın benzetmesi: "Bankada altın hesabın var ama öyle bir altın yok."
- Özyineleme (recursion)
  - Fonksiyonun kendi içinden kendini çağırmasıdır. Bir temel durum (base case) ve kendini tekrar eden adım vardır.
  - Faktöriyel: `if (n == 0 || n == 1) return 1; return n * f(n - 1);`. Döngülü çözümle aynı sayıda çarpma yapar, kod daha kısadır ama her çağrı stack'te yer tutar.
  - Çağrı zinciri: `main` → `f(3)` → `f(2)` → `f(1)`. En son çağrılan ilk tamamlanır, iç içe parantezler gibi. Sonuç geriye doğru toplanır: `1`, `2*1`, `3*2`.

  - Dolaylı özyineleme (indirect recursion): x → y → z → x.
  - Temel durum yoksa çağrılar stack'i doldurur ve işletim sistemi programı kapatır: stack overflow. Hocanın makinesinde yalnızca bir sayaç tutan fonksiyon yaklaşık 43 bin kez, içinde 10 elemanlı `int` dizisi olan fonksiyon yaklaşık 21 bin kez çağrılabildi.
  - Çağrı sayısını saymak için yerel değişken işe yaramaz, her çağrıda yeniden oluşur. `static` değişken (veya global değişken) kullanılır.
  - Fibonacci, döngüyle ve özyinelemeyle yazıldı. Özyinelemeli sürüm her çağrıda iki çağrı yapar. İş yaklaşık 2^{n-1} katına çıkar, aynı değerler tekrar tekrar hesaplanır. Çözümü dinamik programlama (memoization): hesaplanan sonuçlar bir dizide saklanır.
  - Üs alma: $x^n = x^{n/2} \cdot x^{n/2}$ ile $\log_2 n$ çarpmada hesaplanabilir. Derste `return mypower(x, y/2) * mypower(x, y/2);` yazıldı. Hoca kodda bir eksik olduğunu, bunun cuma günü tamamlanacağını söyledi.
- Duyurular: Cuma günü (2 Mayıs) ders online yapılacak. Deprem konulu bir toplantıya Online Yıldız'daki "Toplantılar" modülünden katılınacak. Sınav bilgilendirmesi cuma günü yapılacak.

Hocanın Özellikle Vurguladığı Kısımlar

- Bağlı liste düğümleri mutlaka `malloc` ile heap'te oluşturulmalı. Yerel değişkenin adresini listeye bağlamayın.
- Yeni düğümün `next`'ini `NULL` yapın.
- `head`'i kaybetmeyin. Gezinirken ve silerken geçici işaretçi kullanın.
- `head`'i değiştiren işlemlerde (başa ekleme, baştan silme, listeyi silme) yeni `head` geri döndürülmeli veya `node **` kullanılmalı.
- Global değişkenle çözüm kolay görünür ama tavsiye edilmez.
- Özyinelemede en önemli şey temel durumu doğru belirlemektir.
- Bit alanlarının adresi alınamaz.

Kısa Tekrar Notları

- Düğüm: `typedef struct node { int data; struct node *next; } node;`
- Gezinme: `while (p != NULL) { ...; p = p->next; }`
- Sona ekleme: `while (p->next != NULL) p = p->next; p->next = yeni;`
- Silme: `tmp = head; head = head->next; free(tmp);`
- Araya ekleme: `q->next = p->next; p->next = q;`
- Union boyutu en büyük üye kadar. Bit alanının adresi yok.
- Özyineleme = temel durum + kendini çağıran adım. Temel durum yoksa stack overflow.

Detaylı Açıklamalar

Neden ikinci dereceden işaretçi?

- C'de parametreler değerle aktarılır. `deleteList(node *head)` içinde `head`'e yeni değer atamak `main`'deki `head`'i değiştirmez.
- `main`'deki `head`'i değiştirmek için ya yeni değer döndürülür (`head = deleteList(head);`) ya da `head`'in adresi verilir:

```
/* derste yazılmadı, test edildi */
void deleteList2(node **h) {
    node *t;
    while (*h != NULL) {
        t = *h;
        *h = (*h)->next;
        free(t);
    }
}
/* çağrı: deleteList2(&head);  sonrasında head == NULL */
```

Ders sonrası notlar

- **Kodların kontrolü:** Bağlı liste fonksiyonları (`create`, `addLast`, `printList`, `deleteList`), union örneği, bit alanı ve özyineleme örnekleri gcc ile derlenip denendi. `head = deleteList(head);` sonrası liste "empty" yazar.
- **Union örneğinde `float` değeri:** Derste `c` alanı 0.000000 göründü ve hoca bunu "sıfırlarla dolu" diye açıkladı. Alan tamamen sıfır değildir. 65 tam sayısının bitleri `float` olarak yorumlanınca çok küçük bir sayı (9.1×10^{-44}, denormal) çıkar. `%f` bunu 0.000000 diye yuvarlar. `%g` ile yazdırılırsa görülür. Ayrıca bir üyeye yazıp başka bir üyeden okumak C'de izinlidir ama sonucun anlamı uygulamaya (bayt sırası, `float` biçimi) bağlıdır.
- **Yapı hizalamasının kuralı:** Derste "bellek 8 baytlık hücreler" diye anlatıldı. Kesin kural şu: her üye kendi hizalama gereksinimine (genellikle kendi boyutuna, ör. `int` için 4) uygun adreste başlar, yapının toplam boyutu da en büyük hizalamanın katına yuvarlanır. Örnek (gcc, 64 bit): `struct { char a; short b; char c; int d; }` 12 bayt, aynı üyeler `int, short, char, char` sırasıyla 8 bayt. Büyük üyeleri öne almak genellikle yer kazandırır.
- **Bit alanlarıyla tarih:** `struct { unsigned d:5; unsigned m:4; unsigned y:11; }` gcc'de 4 bayttır. 11 bit en fazla 2047'ye kadar yıl tutar.
- **`mypower` kodundaki eksik:** Derste yazılan `mypower(x, y/2) * mypower(x, y/2)` iki sorun taşır. (1) Tek üslerde yanlış sonuç verir, çünkü `y/2` tam sayı bölmesidir: `mypower(3, 5)` 243 yerine 81 verir. (2) Aynı alt problemi iki kez çağırdığı için toplam çağrı sayısı yine yaklaşık y kadardır, $\log_2 y$'ye inmez. Doğrusu yarıyı bir kez hesaplayıp saklamak ve tek üste bir kez daha x ile çarpmaktır:

```c
/* derste yazılmadı, test edildi: fastpow(3,5)=243 */
long long fastpow(long long x, int y) {
    if (y == 0) return 1;
    long long h = fastpow(x, y / 2);
    return (y % 2) ? h * h * x : h * h;
}
```

- **3 üzeri 45:** Derste 3^{45} hesaplanmak istendi. $3^{45} \approx 2.95 \times 10^{21}$'dir. `int` (en fazla yaklaşık 2.1×10^9) bir yana, 64 bitlik `long long` (en fazla yaklaşık 9.2×10^{18}) bile taşar. Sonuç yanlış çıkar.
- **Fibonacci indeksi:** Derste temel durumlar $F(0) = F(1) = 1$ alındı. Bu durumda dizi 1, 1, 2, 3, 5, 8 olur ve $F(4) = 5$, $F(5) = 8$'dir. Döngülü sürümün ilk hâli 5. terim için 13 verdi. Hoca bunun indeks kaymasından kaynaklandığını gördü ve kodu düzeltti. Döngülü ve özyinelemeli sürümler aynı indekslemeyle aynı sonucu vermelidir.
- **`while (head != NULL)` ile yazdırma:** Bir öğrencinin önerisi doğrudur. Döngü koşulu `head != NULL` olursa son düğüm de döngü içinde yazdırılır, döngüden sonra ayrıca yazdırmaya gerek kalmaz. Derste ilk hâlde koşul `head->next != NULL` olduğu için son eleman ayrıca yazdırılmak zorunda kalmıştı.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.